COS341 2026: Continuation of **Semester-Practical-Specification**
Part 3**a**) ***Code Generation for* ALGO *"in the spirit of" Chapter 6***

General Remarks:
In this phase we assume that the source-program was already type-checked and *not rejected* by the type analyser (from 2b).
In the spirit of Textbook Chapter 6 we now want to **generate code** for the ***Algorithms***, (whereby the problem of how to translate the *Functions* will be dealt with later in the *forthcoming* task sheet 3**b**).
For *this* task sheet 3**a** we note especially:

- Similar to **Fig.6.8** in our book, *we do not want* to use the *Boolean* compositions **and**/**or**/**not** in our *Target-Language*. Thus we will translate SPL's Booleans into *cascading* GOTO jumps
- Different from our textbook, our *target language* in this project does *not* offer any **ELSE** command, such that we will have to generate code for Boolean branching by way of GOTO jumps alone.
- Our target code is *inspired by* some of the *low-level features* of the "classical" programming language ***C*** such that eventually, after the completion of the *forthcoming* Part 3**b**, our ***SPL*** source programs will become actually run-able **:)**

Syntax/Grammar-based ***guidelines* for the translations are given below**, whereby the **green** code is the actual **resulting target code** which must actually get **written out** into the **output file**.
Note: these translation guidelines are giving you only the most important **hints**, but *the* ***details must be figured out by yourself***, at it is *your* project in which *you* must demonstrate *your* ingenuity!

SPL_PROG	→	P$	*We'll deal with this rule later, in the forthcoming part 3**b***
P	→	V_DECL **:** F_DECL **:** ALGO	*We'll deal with this rule later, in the forthcoming part 3**b***
V_DECL	→	ε *// nullable*	*We'll deal with this rule later, in the forthcoming part 3**b***
V_DECL	→	**NAME** V_DECL	*We'll deal with this rule later, in the forthcoming part 3**b***
F_DECL	→	ε *// nullable*	*We'll deal with this rule later, in the forthcoming part 3**b***
F_DECL	→	F_TYPE F_DECL	*We'll deal with this rule later, in the forthcoming part 3**b***
F_TYPE	→	**void NAME (** V_DECL **) { ** P **return }**	*We'll deal with this rule later, in the forthcoming part 3**b***
F_TYPE	→	**num NAME (** V_DECL **) { ** P **return (** TERM **) }**	*We'll deal with this rule later, in the forthcoming part 3**b***
ALGO	→	**ε *// nullable***	***Advice:*** trans(ALGO) = **/* end */**
ALGO	→	INSTR **;** ALGO	***Advice:*** trans(ALGO) = { trans(INSTR) **; ASCII13** trans(ALGO) }

whereby **ASCII13** *is a meta-symbol which represents the new line (or carriage return). In other words: in your output file you'll always write a new line after having written a semi-colon; in this manner the output file will become "neat and legible" for the reader for the purpose of "inspection".*

Turn the page ►

INSTR	→	**print** OUTP	
OUTP	→	**STRING**	***Advice:*** trans(INSTR) = **std::cout << "** STRING **"**

*In other words: We **display** the **given string onto the** computer **screen** by way of the standard **cout** command, which is a standard feature of the "classical" language **C**, which is our target language in this project.*

INSTR	→	**print** OUTP	
OUTP	→	**(TERM)**	***Advice:** we translate the term, 'catch' its value in a new variable, and display the new variable onto the screen. The "translation idea" is similar to: Textbook **Fig.6.3***

trans(INSTR) = { Preparation:
vN ← newly generated varName
code ← **trans(TERM , vN)**
Write out:
typ vN ; ASCII13
code **;** ASCII13
std::cout << vN
}

Additional explanation: Because our target language, ***C***, is meant to be executable, *we must create a valid type declation for each automatically newly generated variable* name–*otherwise* the ***C*** runtime environment *would not accept* our generated target code**!** Thereby, **typ** *depends on what the type analyser has found out in the foregoing Phase* 2**b** of the project:

- If the type analyser had found out that the given SPL source program is dealing with *whole* numbers, then **typ** must be **int** in the generated target code.
- If the type analyser had found out that the given SPL source program is dealing with *real* numbers, then **typ** must be **float** in the generated target code.

INSTR	→	**nop**	***Advice:*** trans(INSTR) = **/* nop */**
INSTR	→	**comment STRING**	***Advice:*** trans(INSTR) = **/*** STRING ***/**
INSTR	→	ASSIGN	***Advice:*** trans(INSTR) = trans(ASSIGN)
INSTR	→	BRANCH	***Advice:*** trans(INSTR) = trans(BRANCH)
INSTR	→	LOOP	***Advice:*** trans(INSTR) = trans(LOOP)
INSTR	→	CALL	***Advice:***
CALL	→	**NAME (INPUT)**	*We'll deal with function calls later in the final Phase 3**b** Our method will be "**inlining**" as shown in the Lectures*
INPUT	→	ε *// nullable*	
INPUT	→	TERM INPUT	
ASSIGN	→	**NAME** = TERM	***Advice:*** trans(ASSIGN) = { Preparation: **vN** ← newly generated varName code ← **trans(TERM , vN)** intern ← *lookUp*(NAME ***SymbT***) Write out: **typ vN ;** ASCII13 code **;** ASCII13 intern = vN }

TERM → **NAME**

Advice: *similar as in Textbook* ***Fig.6.3***

trans(TERM , *resultVar* **)** = { Preparation:
intern ← *lookUp*(NAME ***SymbT***)
Write out:
resultVar = intern
}

TERM → **NUM**

Advice:

trans(TERM , *resultVar* **)** = { Write out:
resultVar = NUM
}

Note:
a number in the SPL source code *simply remains the same* num also in the *C* target code.

TERM → CALL

Advice:
*We'll deal with function calls later in the final Phase 3**b***
*Our method will be "**inlining**" as shown in the Lectures*

TERM → **mod (TERM TERM)**
TERM → **div (TERM TERM)**
TERM → **add (TERM TERM)**
TERM → **sub (TERM TERM)**
TERM → **mul (TERM TERM)**

Advice:
Apply your own intelligence and ingenuity *to deal with all these cases as in Figure* ***6.3*** *of the Textbook: Exp1* ***binop*** *Exp2*
Thereby , **add (TERM TERM)**
is equivalent to: Exp1 + TExp2 *in the Textbook.*

For the generation of the operation symbols in the target language C:

- trans(mod) = **%**
- trans(div) = **/**
- trans(add) = **+**
- trans(sub) = **-**
- trans(mul) = *****

TERM → **neg (TERM)**

Advice:
Apply your own intelligence and ingenuity *to deal with all these cases as in Figure* ***6.3*** *of the Textbook:* ***unop*** *Exp1*
Thereby, **neg (TERM)**
is equivalent to: -Exp1 *in the Textbook; that is: the Negation of a number to its negative value.*
For the generation of the operation symbol in the target language C: trans**(neg)** = **-**

Turn the page ►

BRANCH → **if** BOOL **then {** ALGO_*true* **} else {** ALGO_*false* **}**

Advice:
Now we get to a tricky part, because we assume that our Target Langue does not offer an ELSE command: Here our Project differs from the Textbook! We translate according to the following "idea":
We translate the Bool-Code with a conditional GOTO Jump to the True-Code
*We place the False-Code **straight behind** the Bool-Code*
*We place a **GOTO Exit straight behind** the False-Code*
We place a True-Entrance-Label thereafter
We place the True-Code thereafter
*We place the **Exit-Label** thereafter*
With this design-pattern, no ELSE command will be needed in the target code

trans(BRANCH) = { Preparation:
  La ← system-generated ***new numbered label***
  Lb ← system-generated ***new numbered label***
  Lc ← system-generated ***new numbered label***
  decisionCode ← **trans**(BOOL, *La* , *Lb*)
  trueCode ← **trans**(ALGO_*true*)
  falseCode ← **trans**(ALGO_*false*)
Write out:
  /* if */ ASCII13
  decisionCode **;** ASCII13
  Lb **: /* else */** ASCII13
  falseCode **;** ASCII13
  goto *Lc* **;** ASCII13
  La **: /* then */** ASCII13
  trueCode **;** ASCII13
  Lc **: /* continue */** ASCII13
  }

Additional remark:
*The classical language C does indeed offer the **goto** command!*
***Labels** in C are followed by a colon **:** Do not confuse the colon with the semi-colon*

Turn the page ►

LOOP → COND BOOL **do {** ALGO **}**
COND → **until**

Advice:
If the Target Language does not offer any ELSE command, then the Until-Loop is the loop that is most easy for translation: As soon as the Boolean condition is true we make a far-jump to the loop's exit; otherwise we simply fall into the loop's body-algorithm. Therefore:

trans(LOOP) =
trans(**until** BOOL **do {** ALGO **}**) = { Preparation:
ℓs ← system-generated ***new numbered label***
ℓe ← system-generated ***new numbered label***
d ← system-generated ***new numbered label***
decisionCode ← **trans**(BOOL , ℓe , d)
bodyCode ← **trans**(ALGO)
Write out:
ℓs **: /* until */** ASCII13
decisionCode **;** ASCII13
d **: /* do */** ASCII13
bodyCode **;** ASCII13
goto ℓs **;** ASCII13
ℓe **:** ASCII13
/* exit */ ASCII13
}

LOOP → COND BOOL **do {** ALGO **}**
COND → **while**

Advice:
*Translation via **logically equivalent Program Transformation !***
Since we already know the until-translation we can simply apply

trans(LOOP) = trans(**while** BOOL **do {** ALGO **}**) =
trans(until not (BOOL **) do {** ALGO **})**

Turn the page ►

LOOP → **do {** ALGO **}** COND BOOL
COND → **while**

Advice:
If the Target Language does not offer any ELSE command, then the Do-While-Loop is easy for translation, too: As soon as the Boolean condition is true we make a far-jump to the loop's start; otherwise we simply fall out of the loop. Therefore:

trans(LOOP) =
trans(**do {** ALGO **} while** BOOL) = { Preparation:
ld ← system-generated ***new numbered label***
le ← ***system-generated new numbered label***
decisionCode ← **trans**(BOOL , *ld* , *le*)
bodyCode ← **trans**(ALGO)
Write out:
ld **: /* do */** ASCII13
bodyCode **;** ASCII13
/* while */ ASCII13
decisionCode **;** ASCII13
le **: /* exit */** ASCII13
}

LOOP → **do {** ALGO **}** COND BOOL
COND → **until**

Advice:
*Translation via **logically equivalent Program Transformation!***
Since we already know the do-while-translation, we can apply

trans(LOOP) =
trans(**do {** ALGO **} until** BOOL) =
trans(do { ALGO **} while not (** BOOL **))**

BOOL → **eq (** TERM TERM **)**
BOOL → **larger (** TERM TERM **)**
BOOL → **lesser (** TERM TERM **)**

Advice: *These cases are similar to Textbook **Fig.6.6** , in which the sub-terms of a Boolean Term are Number-Terms, However we must now be careful again, because **Fig.6.6** will generate the ELSE command which we do not have in our Project. Instead of "ELSE" we simply "fall to the bottom". Therefore:*

trans(BOOL , ***label*** , ***dummy***) = { Preparation :
x ← system-generated NewVar
y ← system-generated NewVar
code1 ← **trans**(TERM , ***x***)
code2 ← **trans**(TERM , ***y***)
op ← ***see advice given below***
Write out:
code1 **;** ASCII13
code2 **;** ASCII13
if (***x op y*** **) goto** ***label***
}

For op: trans(eq) = `==` , trans(larger) = `>` , trans(lesser) = `<` *in the Target Language C*

BOOL → **not (BOOL)**

Advice:
*As in Textbook **Fig.6.8**, we can now simply **swap** the **dummy** for the **targetLabel**, such that the dummy is now the new jump-target:*

trans(BOOL , *targetLabel* , *dummy*) =
trans(**not (BOOL)** , *targetLabel* , *dummy*) =
trans(BOOL , *dummy* , *targetLabel***)**

BOOL → **or (** BOOL BOOL **)**

Advice:
*Because we have no ELSE in our Target Language, the translation of or is now much **simpler than in Fig.6.8** of our Texbook, because the trans(BOOL) function emits a direct goto jump command to the desired targetLabel. In case that the jump command does not "trigger" we will simply "fall down" into the 2nd parameter of the Boolean **or**. The dummy is not needed here at all; we need the dummy only for generating **not**-code.*

trans(BOOL , ***targetLabel*** , *dummy*) = { trans(BOOL , ***targetLabel*** , *dummy*) **;** ASCII13
trans(BOOL , ***targetLabel*** , *dummy*)
}

BOOL → **and (** BOOL BOOL **)**

Advice:
Now we are again smarter than our textbook **:)** *Because we know that we must jump to the False-dummy if any one of the two Boolean parameters is not true, we can translate easily:*

trans(BOOL , ***targetLabel*** , *dummy*) = {
trans(**not(** BOOL **)** , *dummy* , *targetLabel*) **;** ASCII13
trans(**not(** BOOL **)** , *dummy* , *targetLabel*)
}

TO DO:

- **Implement the code generator** *as far as you can* with *this* given specification sheet.
- *Part 3**b***, with the code generation methods for Function Call, *will follow soon* ...

And now: **HAPPY PROGRAMMING :)**
